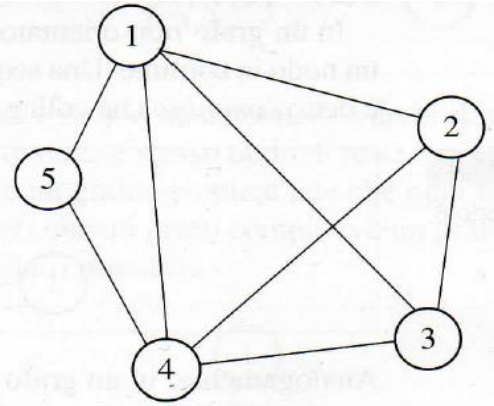
NETWORK OPTIMIZATION

Undirected graph



$G = (N, E)$: pair of sets N and E

$N = \{1, 2, \dots, n\}$: set of **nodes** (or vertices)

$E = \{\dots, \{i, j\}, \dots\}$: set of **links** (or edges)

unordered pair $\{i, j\}$: flow in either direction

Algebraic representation: **incidence matrix**

	{1,2}	{1,3}	{1,4}	{1,5}	{2,3}	{2,4}	{3,4}	{4,5}
1	**1**	**1**	**1**	**1**	0	0	0	0
2	**1**	0	0	0	**1**	**1**	0	0
3	0	**1**	0	0	**1**	0	**1**	0
4	0	0	**1**	0	0	**1**	**1**	**1**
5	0	0	0	**1**	0	0	0	**1**

Incidence matrix ***E***:

- n° of rows = n° of nodes
- n° of columns = n° of links: for each link $\{i, j\}$ the associated column contains
  - entry 1 on the rows associated to nodes i and j (nodes to which the link is incident)
  - entry 0 on all other rows

Star $S(i)$ of node i: set of links incident in node i (links $\{i, j\}$ with entry 1 on row i)

Definition of **tree**:

a graph $T = (N, E_T)$ is a **tree** if

- it is **connected** (= there is a path between each pair of nodes) and
- has **no cycles**

Each of the following conditions is equivalent to the one expressed in the definition,

where $n = |N|$

- T is connected and $|E_T| = n - 1$
- T is cycle-free and $|E_T| = n - 1$
- there is a single path connecting each pair of nodes

Properties of trees

1) A tree with n nodes has $n-1$ links.

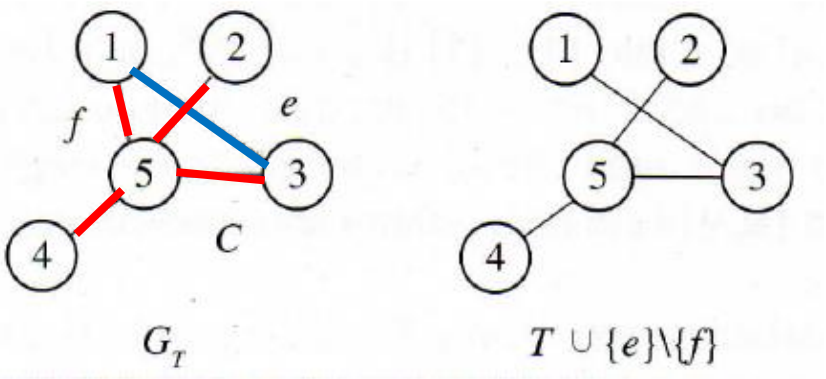
2) There is only one path between any pair of nodes.

3) A cycle is formed by adding to a tree a link which does not belonging to it.

4) Let $T=(N,E_T)$ be a spanning tree of $G=(N,E)$,

   let $e \notin E_T$ be a link not belonging to E_T and let C be the cycle in $E_T \cup \{e\}$:

   for every link $f \in C$, $E_T \backslash \{f\} \cup \{e\}$ is a spanning tree.



Spanning tree and minimum spanning tree (MST)

Spanning tree of the undirected graph $G = (N, E)$:

a subgraph $T = (N, E_T)$ of G (thus with $E_T \subseteq E$) which is a tree

Given an undirected graph $G = (N, E)$ with cost c_e associated to each link $e \in E$,

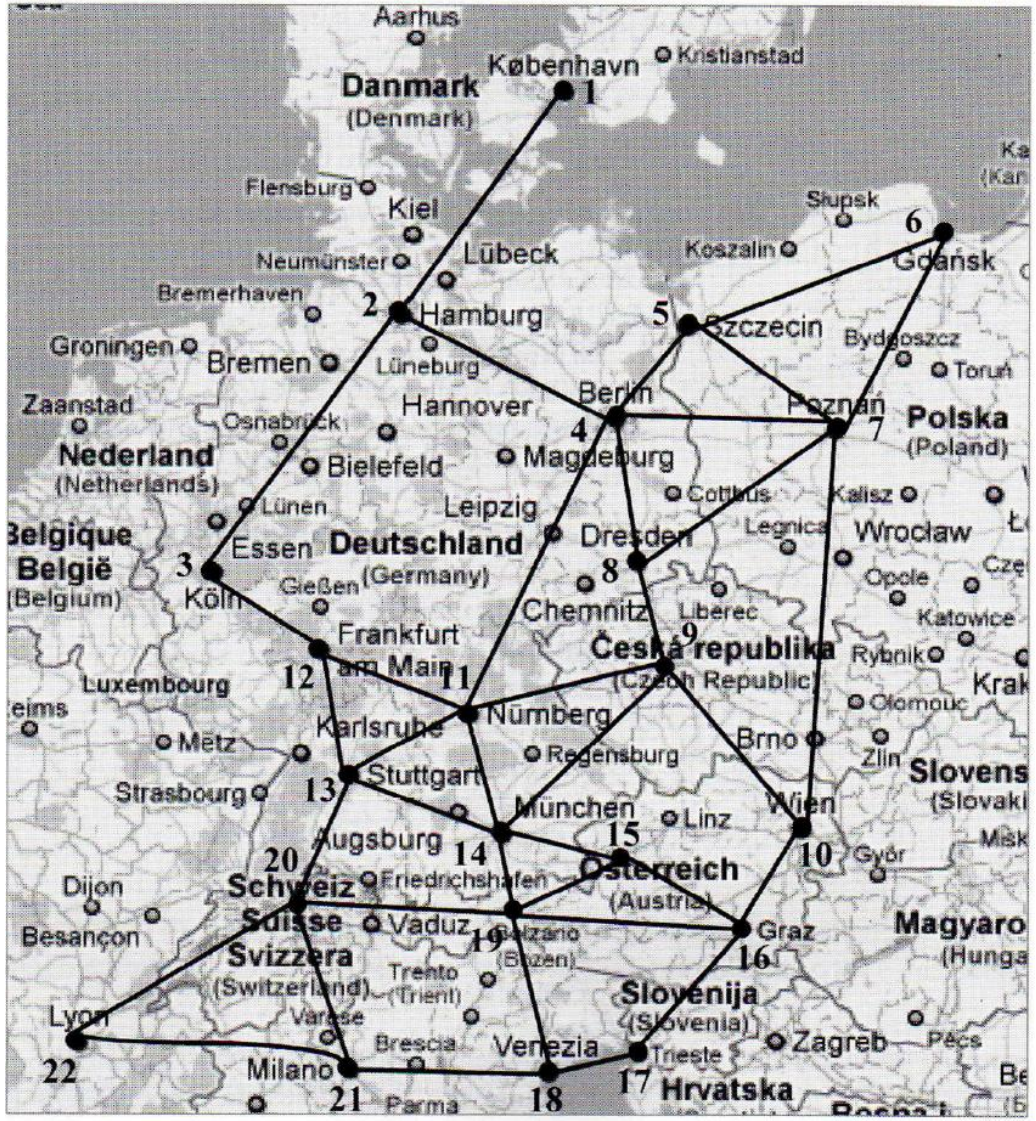
the **minimum spanning tree** problem consists in

finding a spanning tree $T = (N, E_T)$ of G with minimum total cost

$$\min c(T) = \sum_{e \in E_T} c_e$$

Designing a high-speed railway network

between major cities in Central Europe



node	city
1	København
2	Hamburg
3	Köln
4	Berlin
5	Szczecin
6	Gdansk
7	Poznan
8	Dresden
9	Praha
10	Wien
11	Nurnberg
12	Frankfurt am Main
13	Stuttgart
14	München
15	Österreich
16	Graz
17	Trieste
18	Venezia
19	Bolzano
20	Bern
21	Milano
22	Lyon

Lengths of the 38 candidate rail links

link	*km*
{1,2}	204
{2,3}	256
{2,4}	209
{3,12}	115
{4,5}	131
{4,7}	234
{4,8}	149
{4,11}	338
{5,6}	309
{5,7}	226
{6,7}	267
{7,8}	368
{7, 10}	475

link	*km*
{8,9}	101
{9,10}	196
{9,11}	179
{9,14}	227
{10,15}	195
{10,16}	122
{11, 12}	146
{11,13}	120
{11, 14}	94
{12, 13}	133
{13, 14}	131
{14, 15}	86
{14, 19}	95

link	*km*
{15, 16}	170
{15, 19}	111
{16, 17}	229
{16, 19}	318
{17, 18}	102
{18, 19}	192
{18, 21}	155
{19, 20}	194
{20, 13}	152
{20,21}	217
{20,22}	192
{21,22}	211

Undirected graph $G = (N, E)$ with $|N| = n$ and $|E| = m$

- node $\leftrightarrow$ city
- link $\leftrightarrow$ pair of cities connected by a **candidate rail link**

  each link is associated with a cost equal to the distance in km

  between the pairs of cities represented by its vertices

- the constructed railway network must not leave any city isolated

  $\Rightarrow$ find a **connected subgraph** covering all the nodes of G

- the network must be of minimal cost $\rightarrow$ the subgraph must have **no cycles**

  $\Rightarrow$ find a spanning tree $T = (N, E_T)$ of G of minimal cost,

  $\Rightarrow$ i.e. the sum of the costs of its connections is minimal

The Kruskal method (1956)

1) Initialisation:

sort the links in order of non-decreasing cost

2) Iterations:

- among not yet examined links choose one with lowest cost
- insert it in the set E_T^*, if its insertion does not result in a cycle

3) Repeat step 2) until $n - 1$ links are entered

Pseudocode of the Kruskal algorithm

Algorithm Kruskal (N, E, E_T^*)

begin

sort the links in order of non-decreasing cost

Set $E_T^* = \emptyset$;

while $|E_T^*| < n - 1$ **do**

begin

choose a minimum-cost link $e \in E$;

set $E = E \setminus \{e\}$;

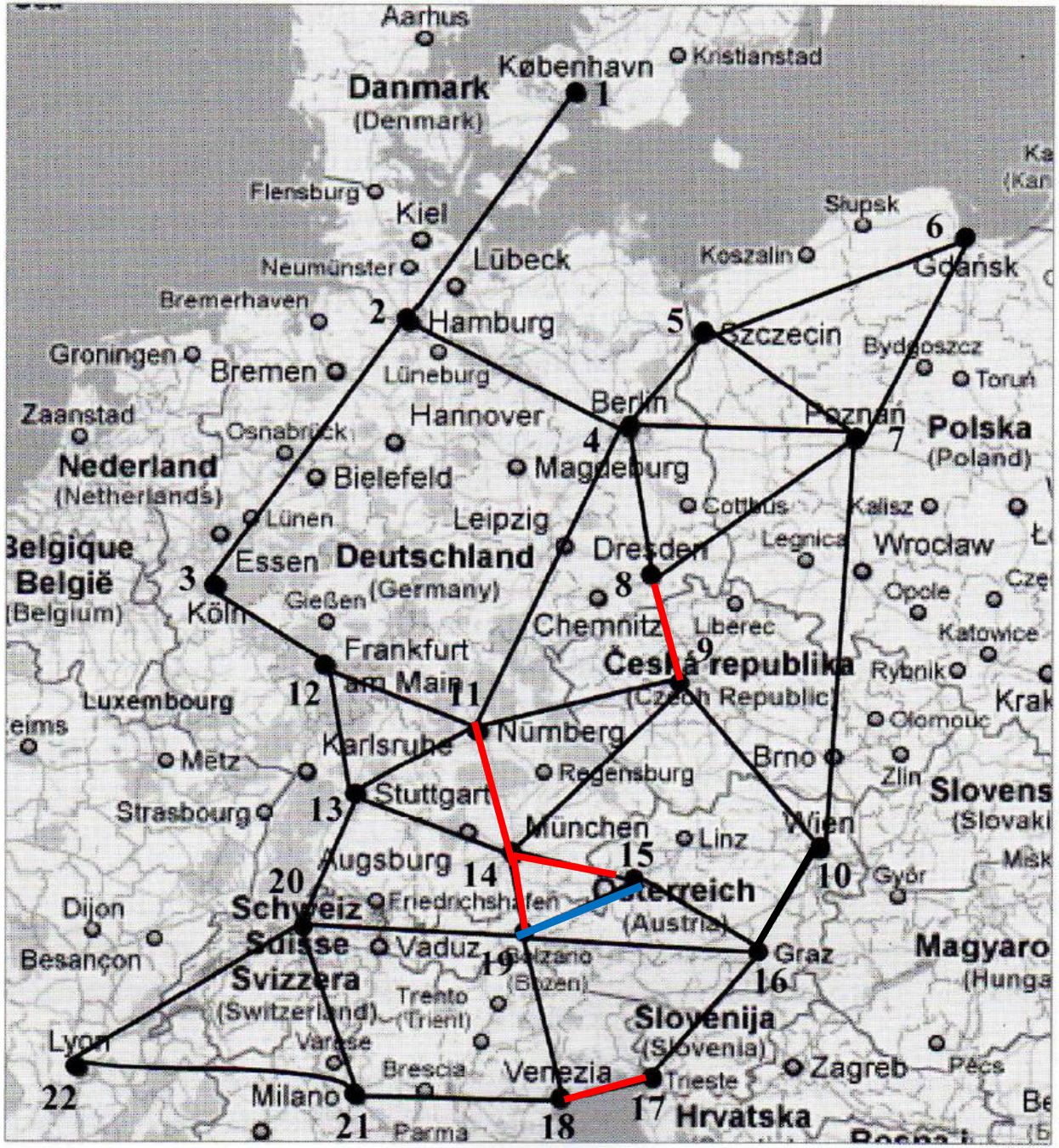
if $E_T^* \cup \{e\}$ is acyclic, **then** $E_T^* = E_T^* \cup \{e\}$;

end

end

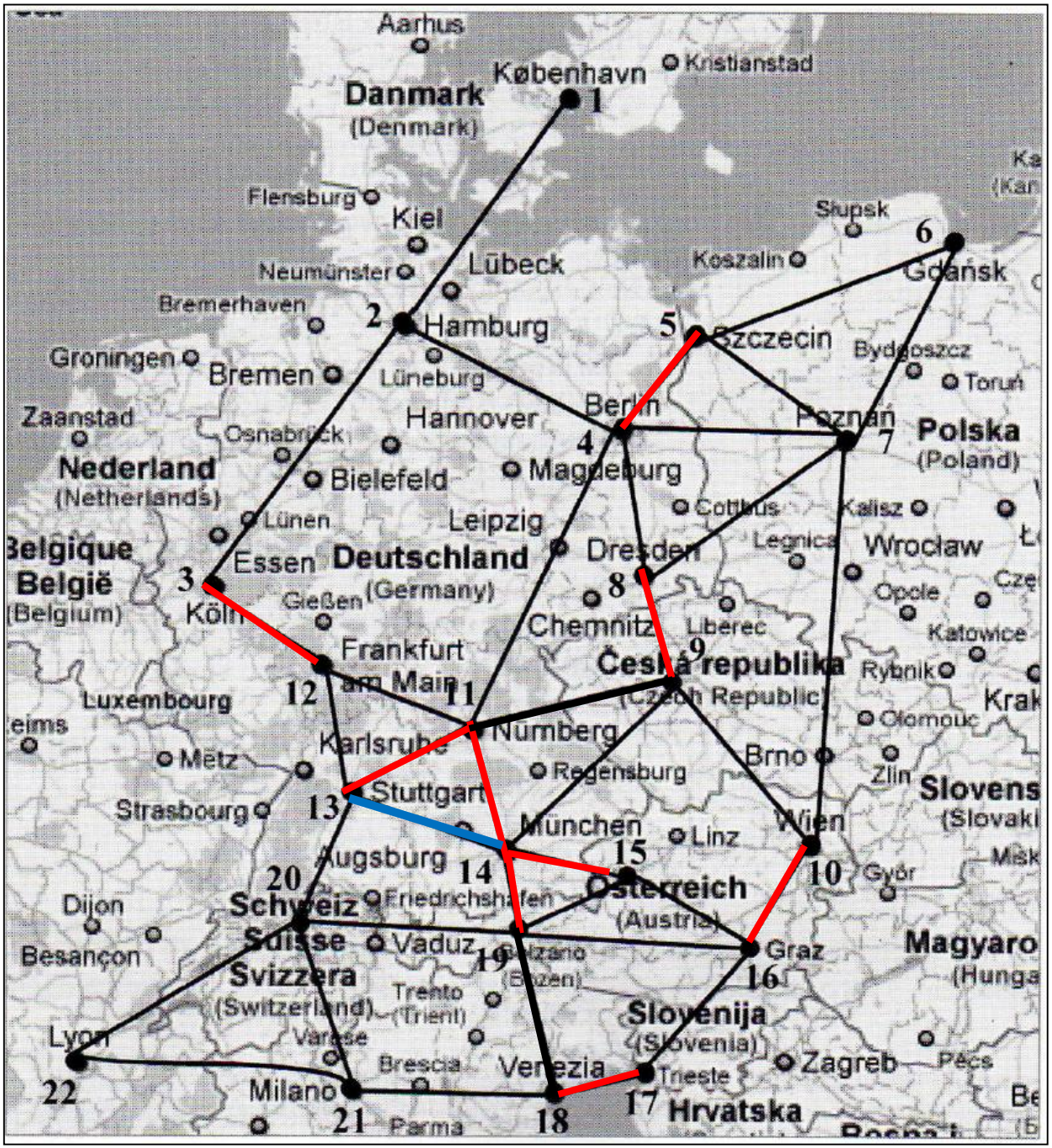
return E_T^*;

end



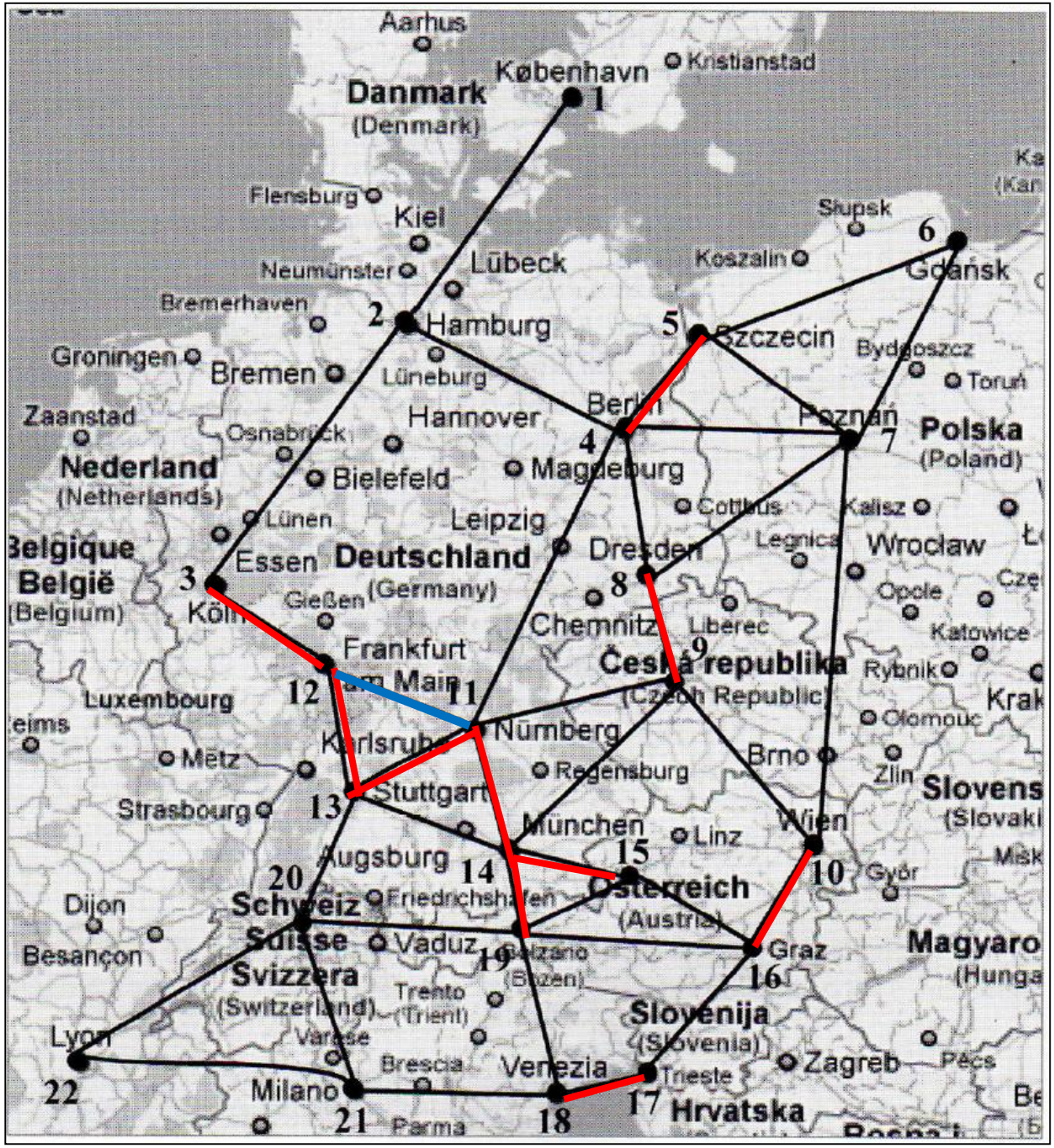
	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
	{3,12}	115
	{11,13}	120
	{10,16}	122
	{4,5}	131
	{13, 14}	131
	{12, 13}	133
	{11, 12}	146
	{4,8}	149
	{20, 13}	152
	{18, 21}	155
	{15, 16}	170
	{9,11}	179
	{18, 19}	192

	link	Km
	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
	{1,2}	204
	{2,4}	209
	{21,22}	211
	{20,21}	217
	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



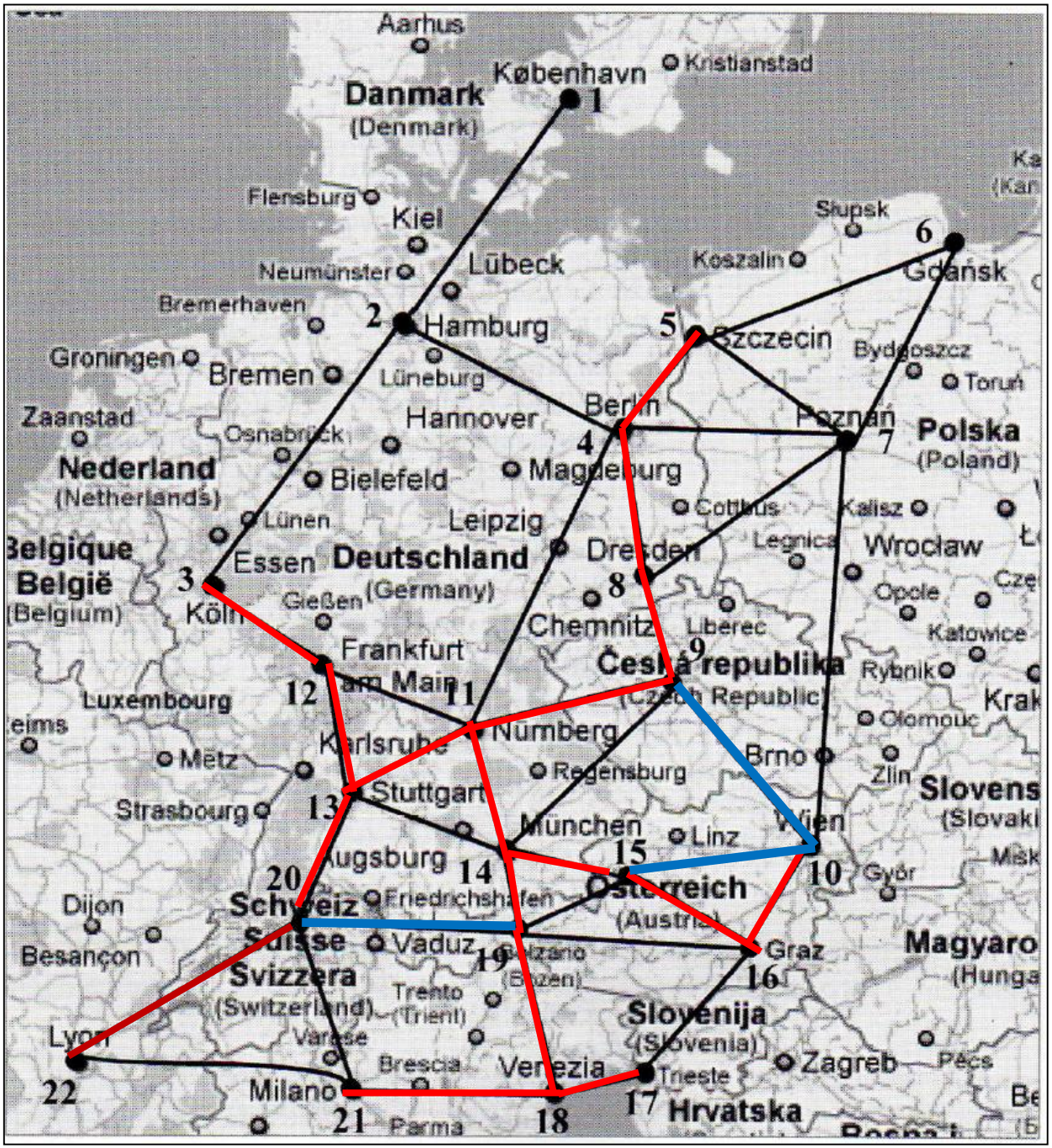
	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
	{12, 13}	133
	{11, 12}	146
	{4,8}	149
	{20, 13}	152
	{18, 21}	155
	{15, 16}	170
	{9,11}	179
	{18, 19}	192

	link	Km
	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
	{1,2}	204
	{2,4}	209
	{21,22}	211
	{20,21}	217
	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



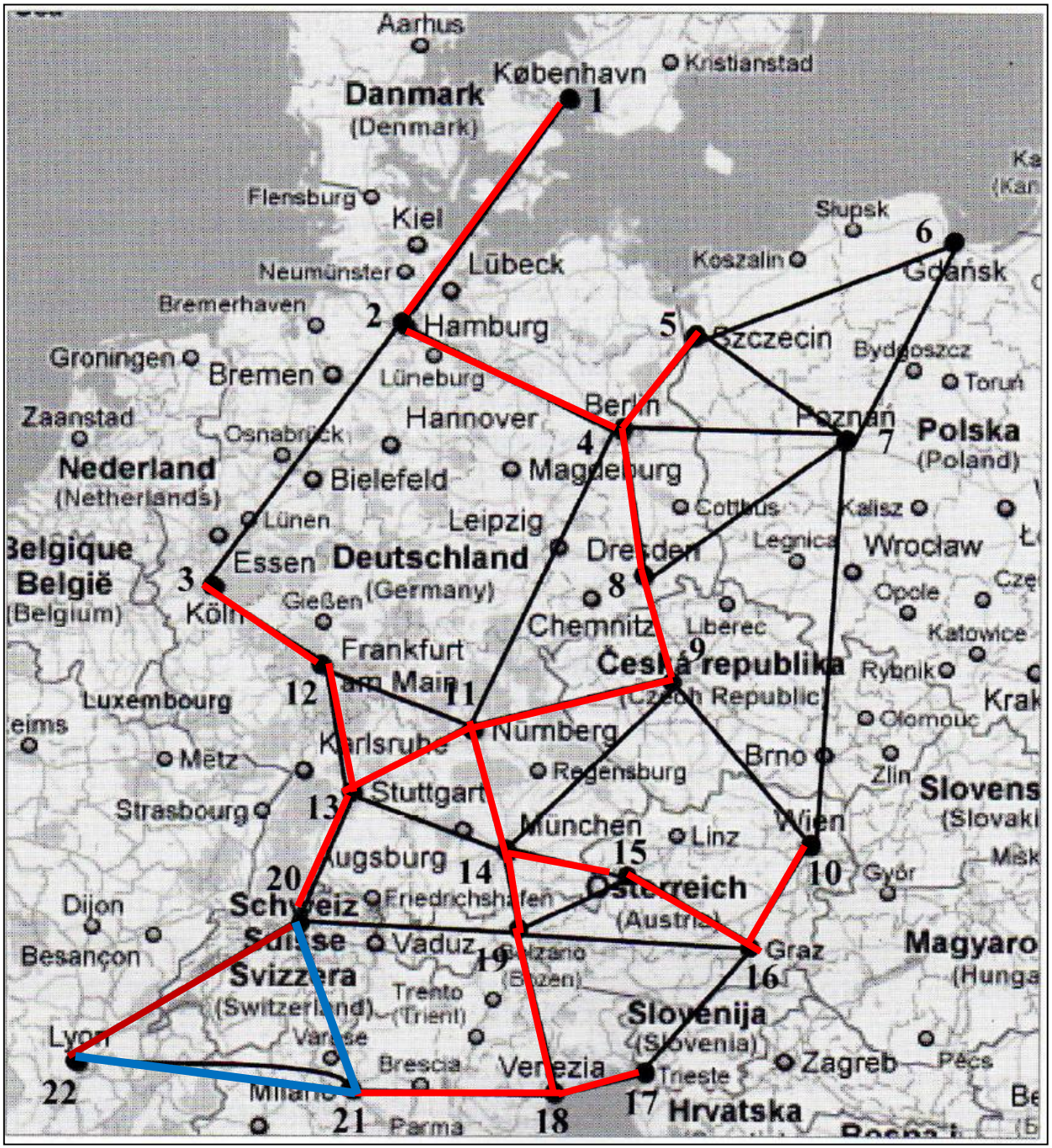
	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
10	{12, 13}	133
	{11, 12}	146
	{4,8}	149
	{20, 13}	152
	{18, 21}	155
	{15, 16}	170
	{9,11}	179
	{18, 19}	192

	link	Km
	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
	{1,2}	204
	{2,4}	209
	{21,22}	211
	{20,21}	217
	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



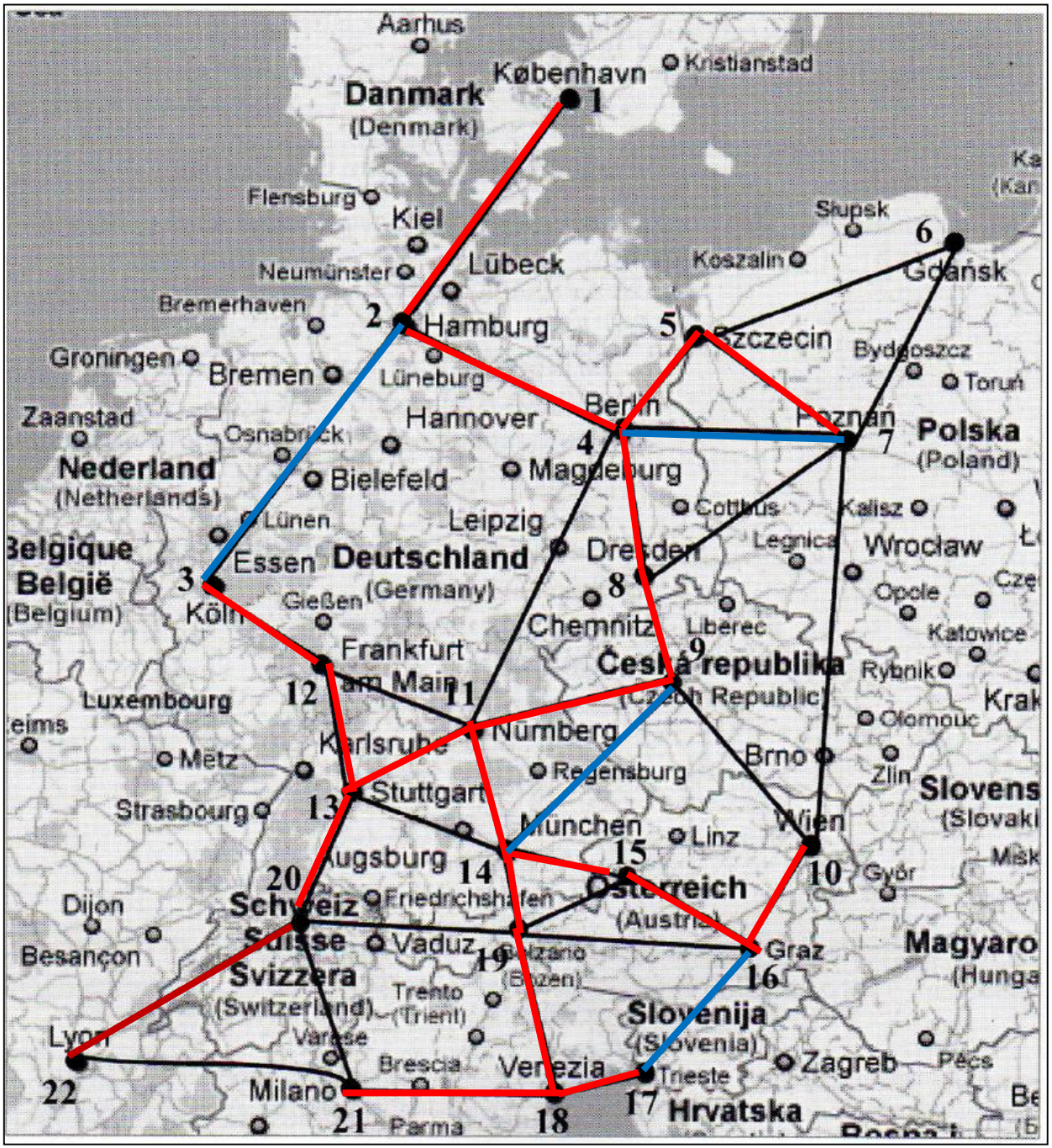
	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
10	{12, 13}	133
	{11, 12}	146
11	{4,8}	149
12	{20, 13}	152
13	{18, 21}	155
14	{15, 16}	170
15	{9,11}	179
16	{18, 19}	192

	link	Km
17	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
	{1,2}	204
	{2,4}	209
	{21,22}	211
	{20,21}	217
	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



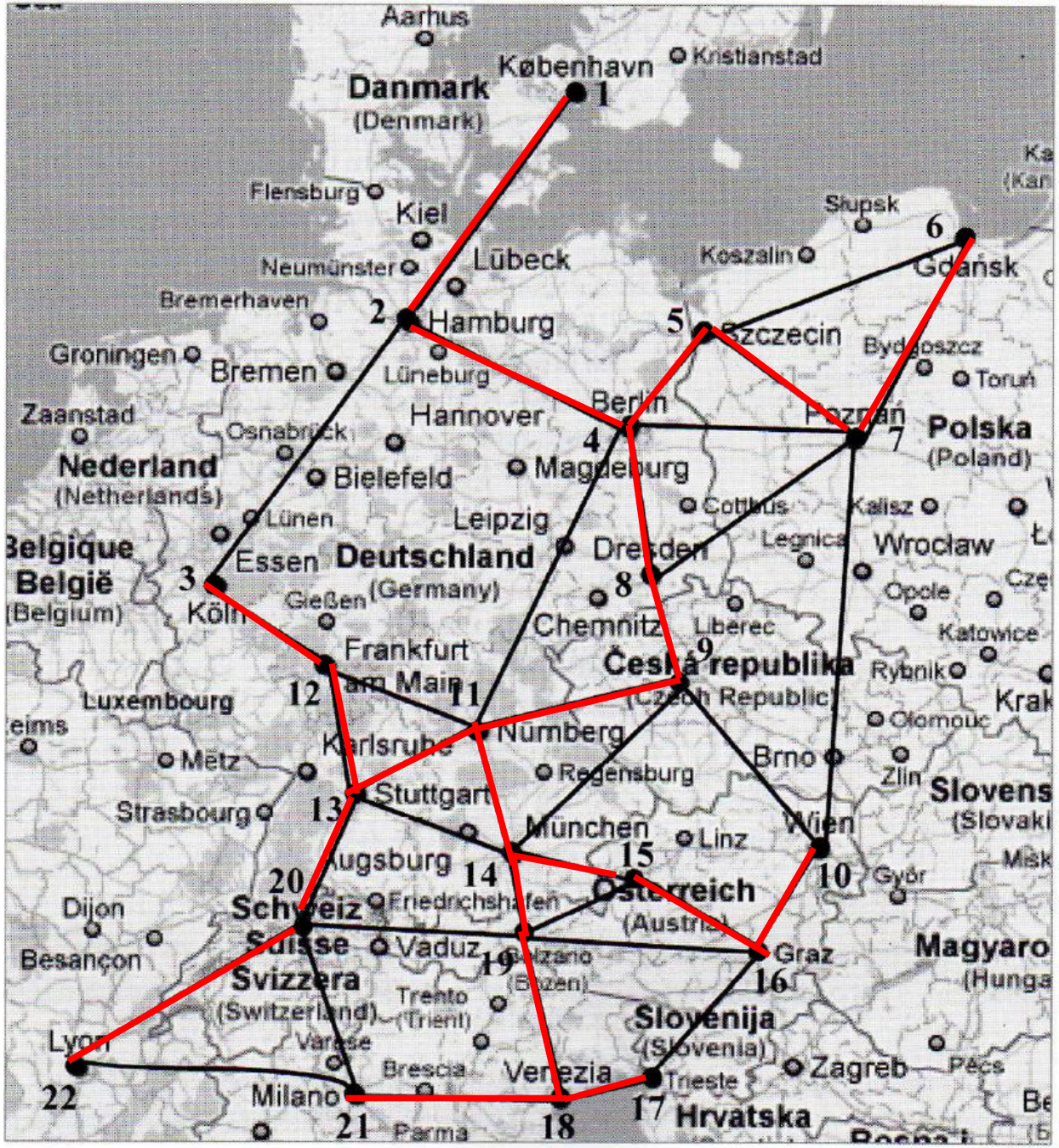
	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
10	{12, 13}	133
	{11, 12}	146
11	{4,8}	149
12	{20, 13}	152
13	{18, 21}	155
14	{15, 16}	170
15	{9,11}	179
16	{18, 19}	192

	link	Km
17	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
18	{1,2}	204
19	{2,4}	209
	{21,22}	211
	{20,21}	217
	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
10	{12, 13}	133
	{11, 12}	146
11	{4,8}	149
12	{20, 13}	152
13	{18, 21}	155
14	{15, 16}	170
15	{9,11}	179
16	{18, 19}	192

	link	Km
17	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
18	{1,2}	204
19	{2,4}	209
	{21,22}	211
	{20,21}	217
20	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475



	link	Km
1	{14, 15}	86
2	{11, 14}	94
3	{14, 19}	95
4	{8,9}	101
5	{17, 18}	102
	{15, 19}	111
6	{3,12}	115
7	{11,13}	120
8	{10,16}	122
9	{4,5}	131
	{13, 14}	131
10	{12, 13}	133
	{11, 12}	146
11	{4,8}	149
12	{20, 13}	152
13	{18, 21}	155
14	{15, 16}	170
15	{9,11}	179
16	{18, 19}	192

	link	Km
17	{20,22}	192
	{19, 20}	194
	{10,15}	195
	{9,10}	196
18	{1,2}	204
19	{2,4}	209
	{21,22}	211
	{20,21}	217
20	{5,7}	226
	{9,14}	227
	{16, 17}	229
	{4,7}	234
	{2,3}	256
21	{6,7}	267
	{5,6}	309
	{16, 19}	318
	{4,11}	338
	{7,8}	368
	{7, 10}	475

Minimum total length: 3041 *km*

The **partial subgraph obtained in intermediate iterations is not connected**:

for example at iteration 4 link {8, 9} is added,

but neither node 8 nor node 9 are connected to any previously chosen link

At iteration 6 link {3, 12} of cost 115 is added, instead of link {15, 19} of cost 111,

because the latter forms a loop with {14, 15} and {14, 19}, previously chosen links.

At iteration 10 link {13, 14} with cost 131 is not inserted.

The Prim algorthm for MST

Algorithm Prim (N, E, E_T^*)

begin

set $E_T^* = \emptyset$;

set $S = \{1\}$;

while $|E_T^*| < n - 1$ **do**

begin

in $\delta(S)$ choose a minimum cost link $\{v, h\} \in \delta(S)$;

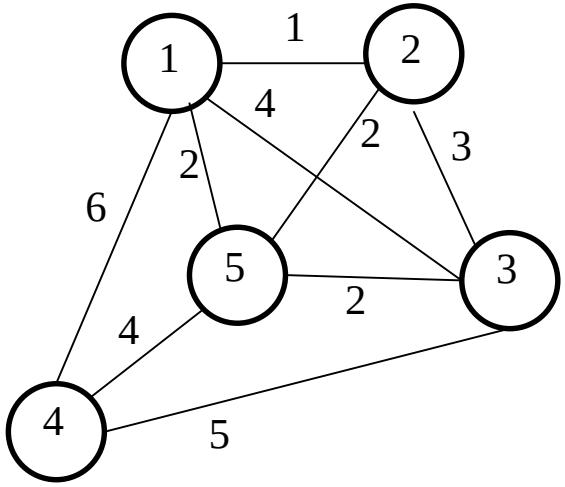
set $E_T^* = E_T^* \cup \{\{v, h\}\}$;

set $S = S \cup \{h\}$;

end

return E_T^*;

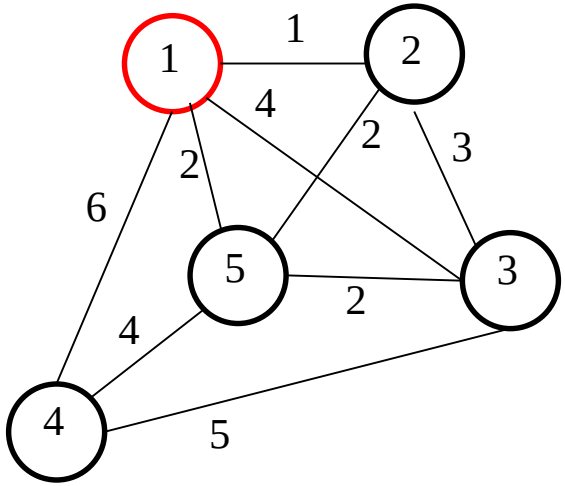
end



$N = \{1, 2, 3, 4, 5\}$

$A = \{\{1,2\}, \{1,3\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,4\}, \{3,5\}, \{4,5\}\}$

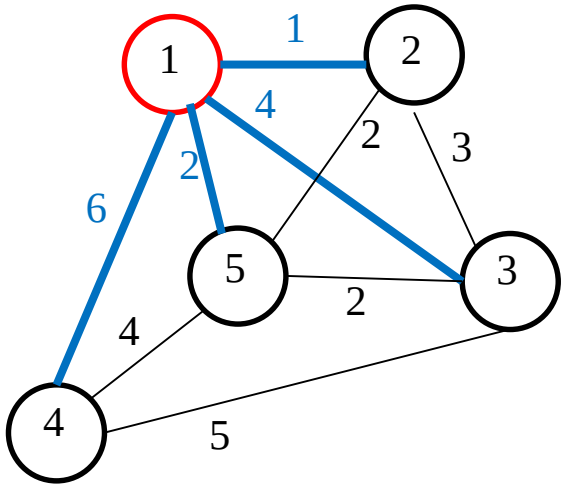
Initialisation: $E_T^* = \emptyset$ and $S = \{1\}$



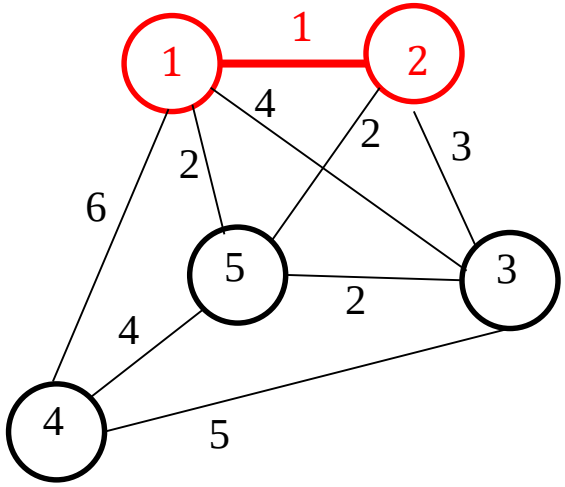
Iteration 1

$N = \{1, 2, 3, 4, 5\}$ $A = \{\{1,2\}, \{1,3\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,4\}, \{3,5\}, \{4,5\}\}$

$S = \{1\}$ and $N \backslash S = \{2, 3, 4, 5\}$ $\rightarrow$ cut links $\delta(S) = \{\{1,2\}, \{1,3\}, \{1,4\}, \{1,5\}\}$.



Link of minimum cost: $\{1,2\}$ with cost 1.



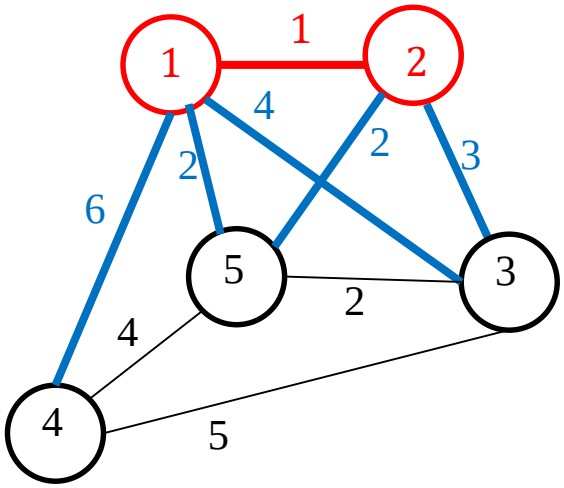
Set $E_T^* = \{\{1,2\}\}$ and $S = \{1,2\}$.

Since $|E_T^*| = 1 < n - 1 = 4$, the procedure continues.

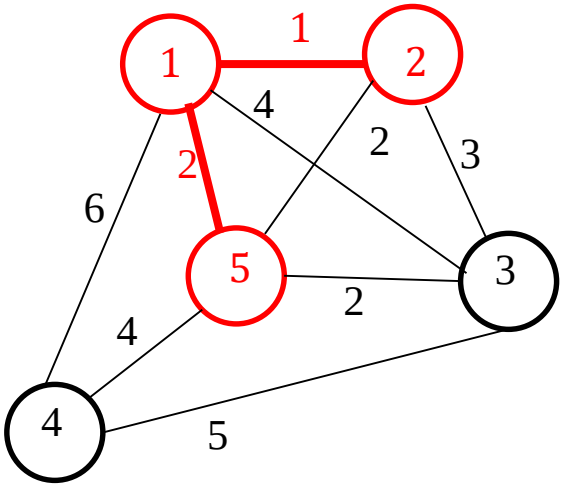
Iteration 2

$N = \{1, 2, 3, 4, 5\}$ $\quad A = \{\{1,2\}, \{1,3\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,4\}, \{3,5\}, \{4,5\}\}$

$S = \{1, 2\}$ and $N \backslash S = \{3, 4, 5\}$ $\quad \rightarrow \quad$ cut links $\delta(S) = \{\{1,3\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,5\}\}$



Link of minimum cost: $\{1, 5\}$ with cost 2.



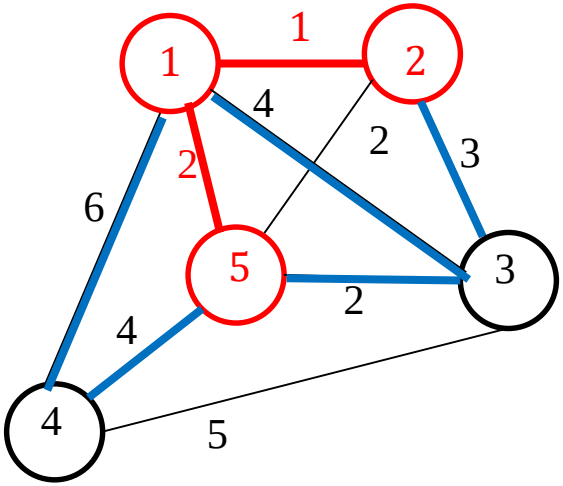
Set $E_T^* = \{\{1, 2\}, \{1, 5\}\}$ and $S = \{1, 2, 5\}$.

Since $|E_T^*| = 2 < n - 1 = 4$, the procedure continues.

Iteration 3

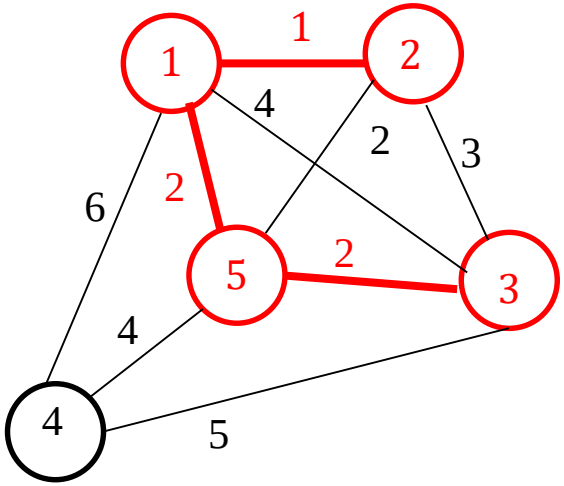
$N = \{1, 2, 3, 4, 5\}$ $A = \{\{1,2\}, \{1,3\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,4\}, \{3,5\}, \{4,5\}\}$

$S = \{1, 2, 5\}$ and $N \backslash S = \{3, 4\}$ $\rightarrow$ cut links $\delta(S) = \{\{1,3\}, \{1,4\}, \{2,3\}, \{5,3\}, \{5,4\}\}$



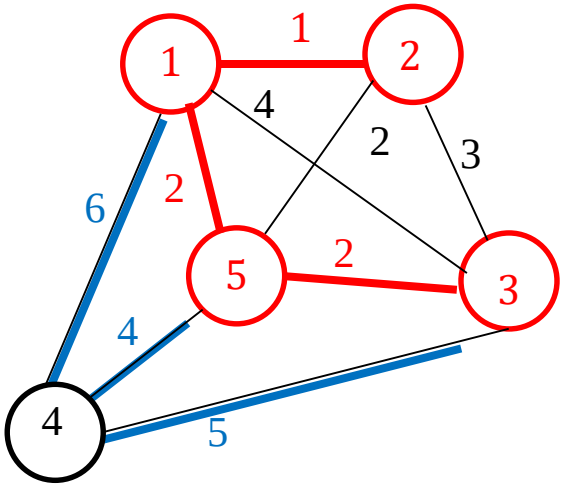
Link of minimum cost: $\{5, 3\}$ with cost 2.

Set $E_T^* = \{\{1,2\}, \{1,5\}, \{5,3\}\}$ and $S = \{1, 2, 3, 5\}$.



Since $|E_T^*| = 3 < n - 1 = 4$, the procedure continues.

Iteration 4



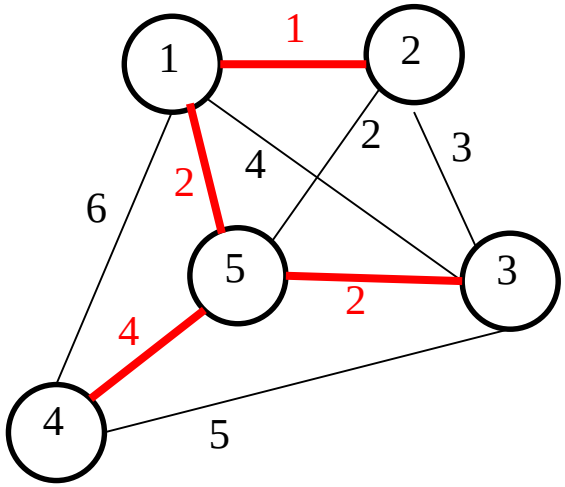
With $S = \{1, 2, 3, 5\}$ the cut links $\delta(S)$ are $\{\{1, 4\}, \{5, 4\}, \{3, 4\}\}$.

Minimum cost link: $\{5, 4\}$ with cost 4.

Set $E_T^* = \{\{1, 2\}, \{1, 5\}, \{5, 3\}, \{5, 4\}\}$ and $S = \{1, 2, 3, 4, 5\}$.

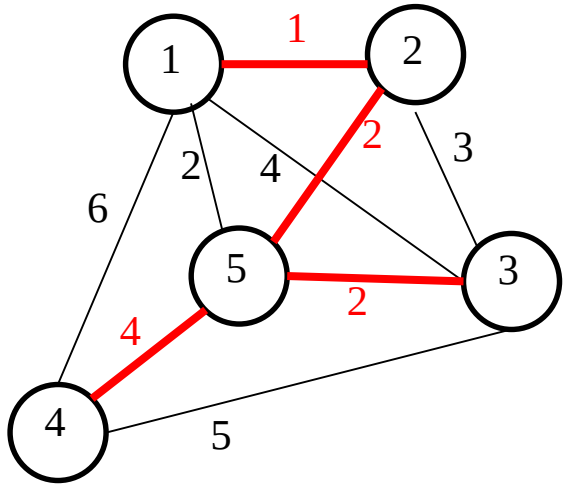
Since $|E_T^*| = 4$, the procedure terminates.

Multiple optima



$E_T^* = \{\{1,2\},\{\mathbf{1},\mathbf{5}\},\{5,3\},\{5,4\}\}$

$c(T) = 9$



$E_T^* = \{\{1,2\},\{\mathbf{2},\mathbf{5}\},\{5,3\},\{5,4\}\}$

$c(T) = 9$

NOTE:

In intermediate iterations of Prim's algorithm

the partial subgraph is always connected,

unlike the Kruskal algorithm.

Definition of **cost reducing link**

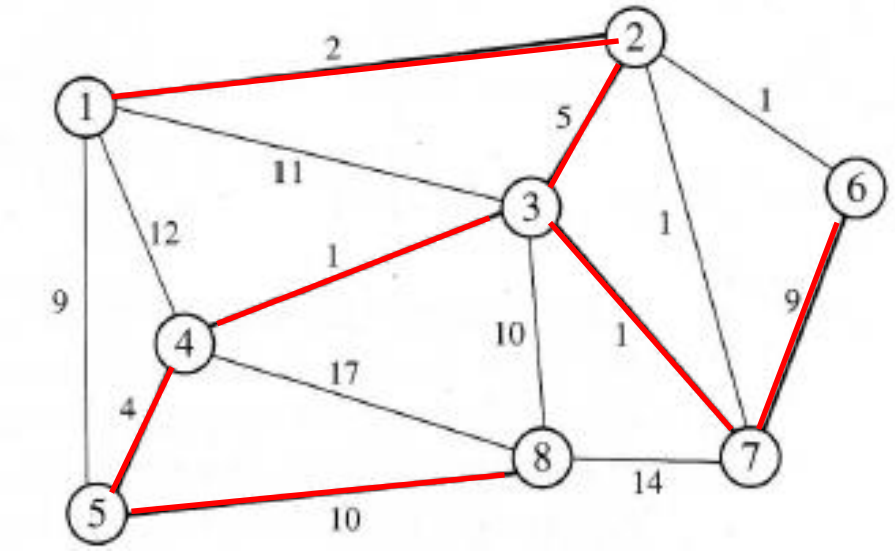
Let $T = (N, E_T)$ be a spanning tree of $G = (N, E)$.

The link $e \notin E_T$ is a **cost reducing link** of T,

if in the cycle C formed by adding e to E_T a link $f \in C \backslash \{e\}$ exists with larger cost ($c_f > c_e$).

Tree optimality condition

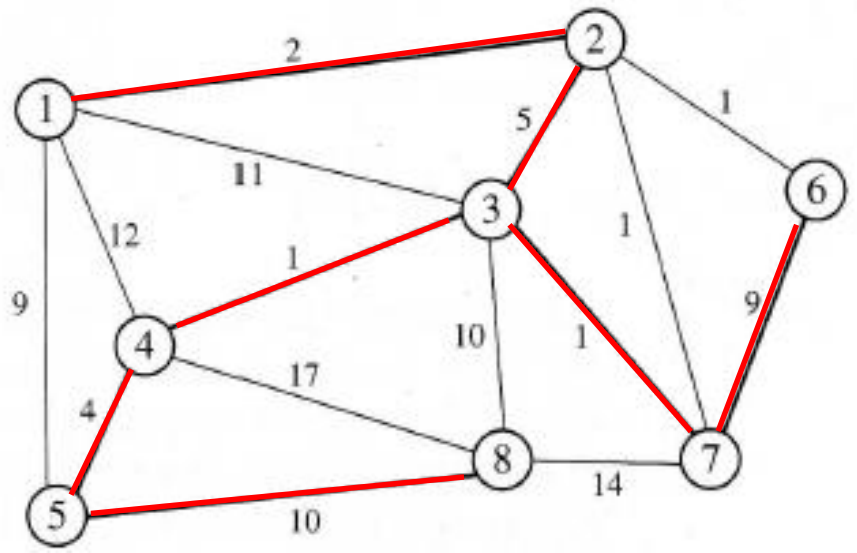
Spanning tree $T = (N, E_T)$ of $G = (N, E)$ has minimal cost ***if and only if*** **no cost reducing link exist**.



Optimality check

Check whether spanning tree {{1, 2}, {2, 3}, {3, 4}, {3, 7}, {4, 5}, {5, 8}, {7, 6}} is of minimal cost.

For the tree optimality condition it is sufficient to verify that the tree has no cost reducing links.



For example, by adding link {1, 5} (which does not belong to the tree) the cycle 1-2-3-4-5-1 is formed,

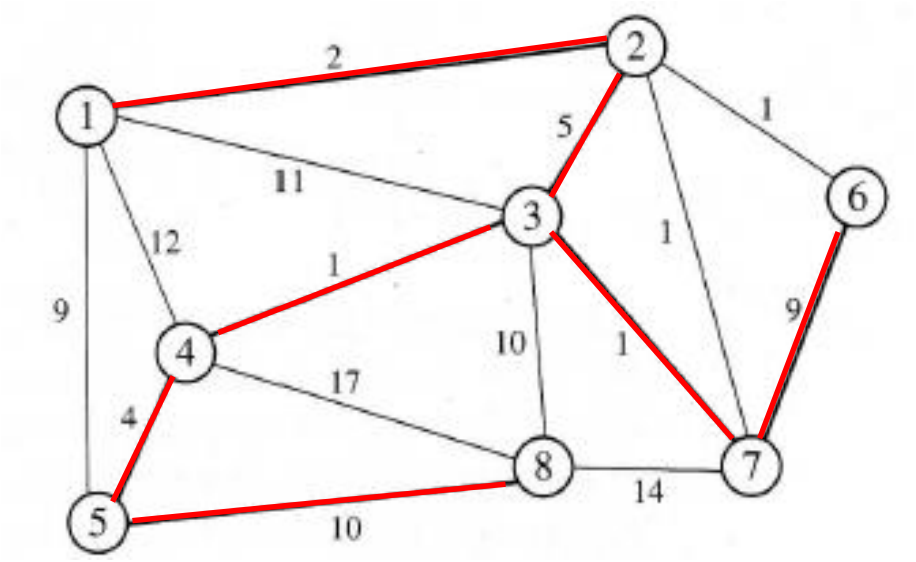
in which no link has greater cost than the cost of {1, 5},

so {1, 5} is not a cost reducing link

Also by adding link {3, 8} (not belonging to the tree) the cycle 3-8-5-4-3 is formed,

in which no link has greater cost than the cost of {3, 8},

therefore {3, 8} is not a cost reducing link



However, the addition of link {2, 6} forms cycle 2-6-7-3-2

in which two links have cost greater than the cost of {2, 6},

therefore {2, 6} is a cost reducing link and the given spanning tree in non optimal.

Note

The **number of checks to be carried out** to detect the existence of a cost reducing link

- in the worst case is $m - (n - 1)$, where n is the number of nodes and m is the number of links

- depends on the order in which we consider the links that are not part of the tree:

  rule of thumb: : examine links in ascending order of cost

Application: optimal transmission of a message

Let $G = (N, E)$ represent a communication network.

The link $\{i, j\} \in E$ indicates that nodes i and j communicate directly:

$p_{i,j}$ is the (known) probability that the message is intercepted on link $\{i, j\}$.

Determine the connections that

allow messages to be transmitted between any pair of nodes

while minimising the probability to be intercepted.

- Messages are to be transmitted between any pair of nodes

  $\Rightarrow$ determine a subgraph which is **connected** and **covers all nodes**

- To minimise the probability for the message to be intercepted, redundancies must be avoided

  $\Rightarrow$ determine a subgraph **without cycles**

$\Rightarrow$ The subgraph to be determined is a **spanning tree** T of graph G

Among all the spanning trees of G we want to determine one with the lowest interception probability

$$\min \prod_{\{i,j\} \in T} p_{i,j}$$

Target: minimise the **product of the weights** of the links chosen to form T.

We want to re-express the objective so that it uses **sums** instead of **products**:

- we express the objective in terms of **maximising the probability that the message <u>will not be intercepted</u>** on the links where it is sent

$$\max \prod_{\{i,j\}\in T} \left(1 - p_{i,j}\right)$$

- using the logarithm function, which is monotonically increasing, the objective is expressed in the form

$$\max \log \prod_{\{i,j\}\in T} \left(1 - p_{i,j}\right) = \max \sum_{\{i,j\}\in T} \log\left(1 - p_{i,j}\right)$$

This transformation does not change the optimal solutions, it only changes the o.f. value

In this way, the problem has been stated as a **spanning tree of maximal value**, where

$$\log\left(1 - p_{i,j}\right)$$

is the value associated to link $\{i, j\} \in E$.

This problem can be solved by means of the Kruskal algorithm or of the Prim algorithm,

suitably adapted for determining the **maximum-value support tree** (instead of minimum cost)

Kruskal algorithm for the **maximum value support tree**

Algorithm Kruskal_max (N, E, E_T^*)

begin

sort the links in order of **non-increasing** **value**

set $E_T^* = \emptyset$;

while $|E_T^*| < n - 1$ **do**

begin

choose a link $e \in E$ of **maximum** **value**;

set $E = E \setminus \{e\}$;

if $E_T^* \cup \{e\}$ is acyclic, **then** $E_T^* = E_T^* \cup \{e\}$;

end

end

return E_T^* ;

end

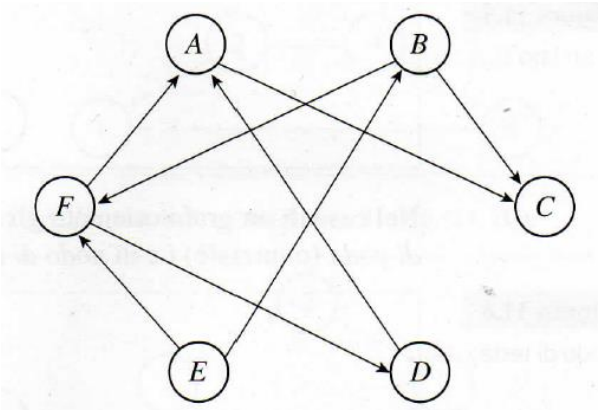
Directed graph

$G = (N, A)$: set of sets N and A

$N = \{1, 2, \dots, n\}$: set of **nodes**

$A = \{\dots, (i, j), \dots\}$: set of **arcs**

ordered pair (i, j): the flow must go from node i to node j



Algebraic representation: **incidence matrix**

	(A,C)	(B,C)	(B,F)	(D,A)	(E,B)	(E,F)	(F,A)	(F,D)
A	-1	0	0	1	0	0	1	0
B	0	-1	-1	0	1	0	0	0
C	1	1	0	0	0	0	0	0
D	0	0	0	-1	0	0	0	1
E	0	0	0	0	-1	-1	0	0
F	0	0	1	0	0	1	-1	-1

Forward star $FS(i)$ of node i: set of arcs leaving node i $\quad FS(i) = \{(i, j) \in A\}$

Backward star $BS(i)$ of node i: set of arcs entering node i $\quad BS(i) = \{(j, i) \in A\}$

Incidence matrix $\boldsymbol{E}$:

- as many lines as nodes
- as many columns as arcs: for each arc the associated column contains
  - entry -1 on the row associated to the node from which the arc exits
  - entry 1 on the row associated to the node in which the arc enters
  - entry 0 in all other rows

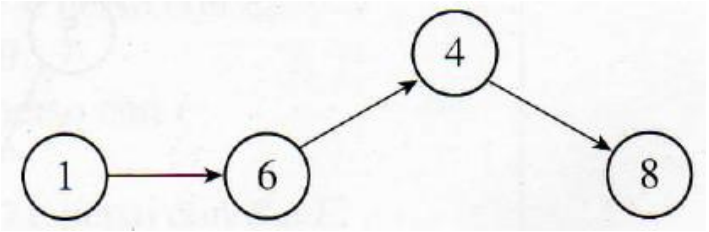
	(A,C)	(B,C)	(B,F)	(D,A)	(E,B)	(E,F)	(F,A)	(F,D)
(node A)	-1	0	0	1	0	0	1	0
(node B)	0	-1	-1	0	1	0	0	0
(node C)	1	1	0	0	0	0	0	0
(node D)	0	0	0	-1	0	0	0	1
(node E)	0	0	0	0	-1	-1	0	0
(node F)	0	0	1	0	0	1	-1	-1

Paths

In a **directed graph**, a path is a sequence of arcs in which

the terminal node of each arc coincides with the initial node of the following arc

Path from node 1 to node 8



Problem:

given a **directed graph** $G = (N, A)$ and

a **root node** r,

determine the set of nodes t that can be **reached** from r,

i.e. in graph $G = (N, A)$ a path from s to t exists.

Input :

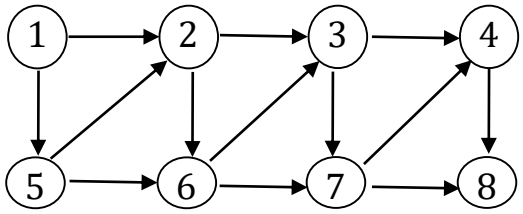
- directed graph $G = (N, A)$
- the terminal nodes of the **forward star** of node $n \in N$ **are in increasing order**
- node r

Structures used in the algorithtm;

- the list Q contains the nodes that have been reached (it is managed as **either a queue or a stack**)
- vector $\boldsymbol{p}$: the value of the component p_j indicates the node precedeeing node j in the path

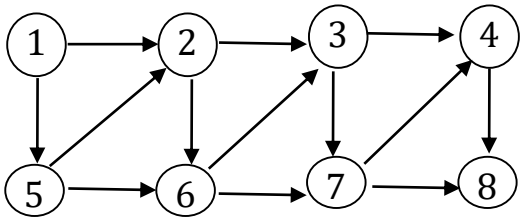
  (if $p_j = 0$, node j has no predecessor, i.e. there is no path from the root to node j)

Algorithm with Q organized as a **queue** (fila) – Initialization



$p_1 = nil \quad p_2 = 0 \quad p_3 = 0 \quad p_4 = 0$

$Q = \{1\}$



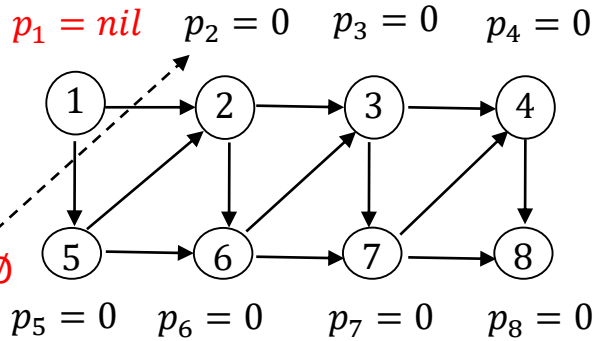
$p_5 = 0 \quad p_6 = 0 \quad p_7 = 0 \quad p_8 = 0$

Version with Q organized as a **queue** (fila) – Iteration 1

$Q = \{1\}$

Selected node: $i = 1$

Delete the selected node from $Q \rightarrow Q = \emptyset$



Forward star of the selected node:

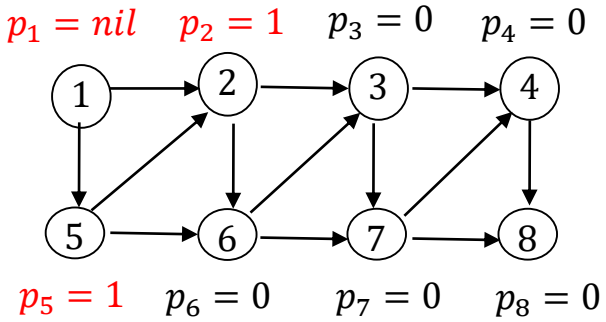
$FS(1) = \{(1,2), (1,5)\}$

$(1,2)$: $p_2 = 0$: update p_2 and Q

$\Rightarrow \quad p_2 = 1 \qquad Q = \{2\}$

$(1,5)$: $p_5 = 0$: update p_5 and Q

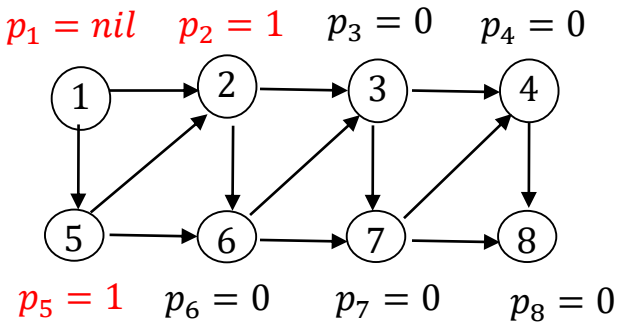
$\Rightarrow \quad p_5 = 1 \qquad Q = \{2,5\}$



Version with Q organized as a **queue** (fila) – Iteration 2

$Q = \{2, 5\}$

Selected node: $i = 2 \rightarrow Q = \{5\}$



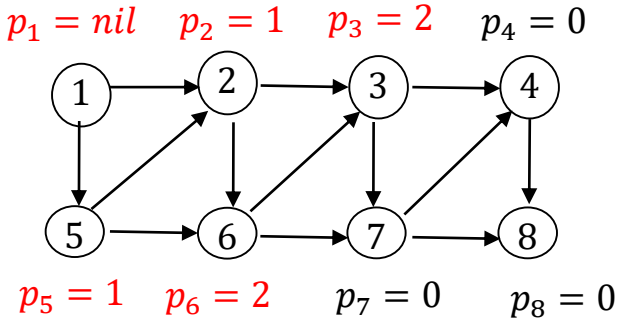
$FS(2) = \{(2, 3), (2, 6)\}$

$(2, 3)$: $p_3 = 0$

$\Longrightarrow \quad p_3 = 2 \qquad Q = \{5, 3\}$

$(2, 6)$: $p_6 = 0$

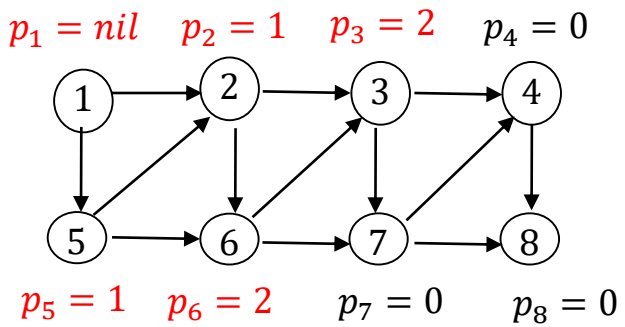
$\Longrightarrow \quad p_6 = 2 \qquad Q = \{5, 3, 6\}$



Version with Q organized as a **queue** (fila) – Iteration 3

$Q = \{5, 3, 6\}$

Selected node: $i = 5 \rightarrow Q = \{3, 6\}$



$FS(5) = \{(5, 2), (5, 6)\}$

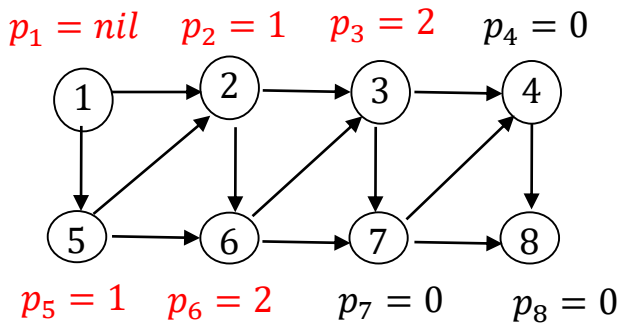
$(5, 2)$: $p_2 \neq 0$: **no update**

$(5, 6)$: $p_6 \neq 0$: **no update**

Version with Q organized as a **queue** (fila) – Iteration 4

$Q = \{3, 6\}$

Selected node: $i = 3 \rightarrow Q = \{6\}$



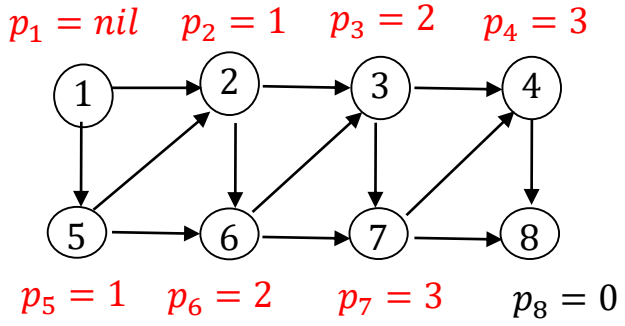
$FS(3) = \{(3, 4), (3, 7)\}$

$(3, 4)$: $p_4 = 0$

$\Rightarrow \quad p_4 = 3 \qquad Q = \{6, 4\}$

$(3, 7)$: $p_7 = 0$

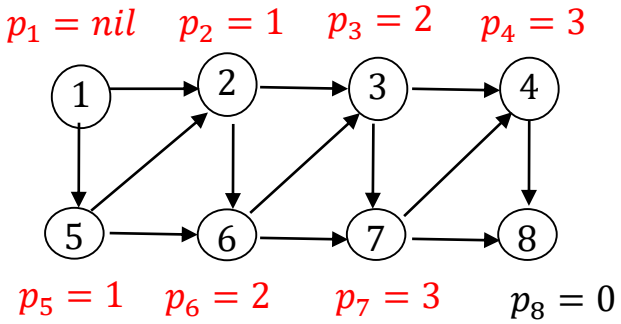
$\Rightarrow \quad p_7 = 3 \qquad Q = \{6, 4, 7\}$



Version with Q organized as a **queue** (fila) – Iteration 5

$Q = \{6, 4, 7\}$

Selected node: $i = 6 \rightarrow Q = \{4, 7\}$



$FS(6) = \{(6, 3), (6, 7)\}$

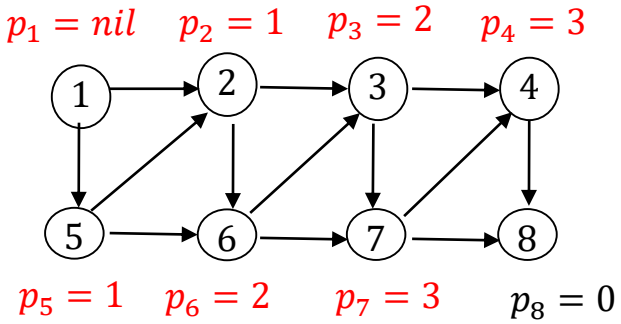
$(6, 3)$: $p_3 \neq 0$

$(6, 7)$: $p_7 \neq 0$

Version with Q organized as a **queue** (fila) – Iteration 6

$Q = \{4, 7\}$

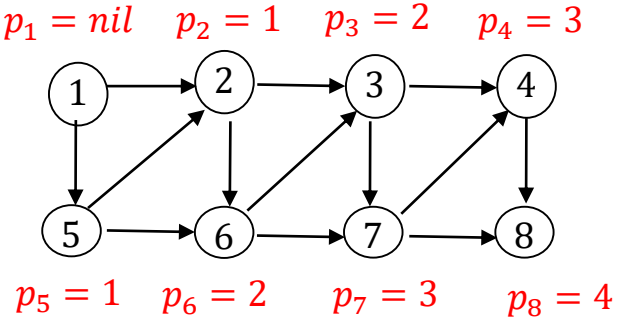
Selected node: $i = 4 \rightarrow Q = \{7\}$



$FS(4) = \{(4, 8)\}$

$(4, 8)$: $p_8 = 0$

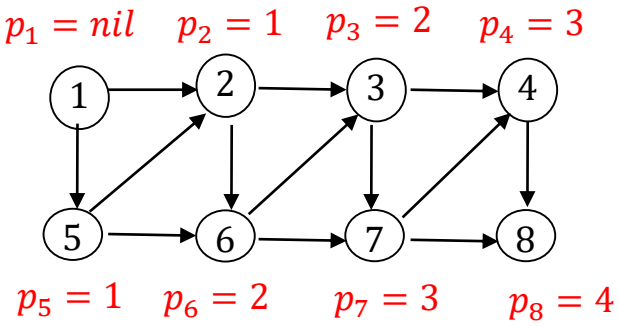
$\Longrightarrow \quad p_8 = 4 \qquad Q = \{7, 8\}$



Version with Q organized as a **queue** (fila) – Iteration 7

$Q = \{7, 8\}$

Selected node: $i = 7 \rightarrow Q = \{8\}$



$FS(7) = \{(7, 4), (7, 8)\}$

$(7, 4)$: $p_4 \neq 0$

$(7, 8)$: $p_8 \neq 0$

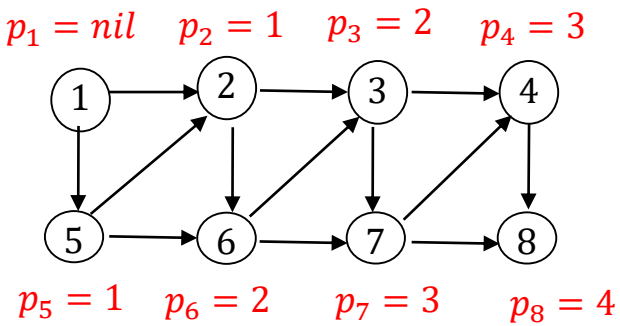
Version with Q organized as a **queue** (fila) – Iteration 8

$Q = \{8\}$

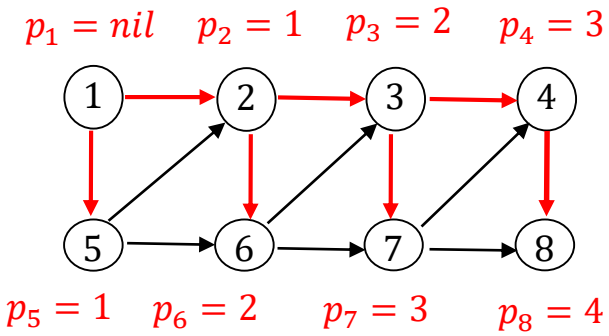
Selected node: $i = 8 \rightarrow Q = \emptyset$

$FS(8) = \emptyset$

$Q = \emptyset$: the algorithm terminates.



The tree of paths obtained when list Q is a **queue**



All nodes can be reached from the root node $r = 1$, since $p_j \neq 0$ for all nodes j

- from node 1 to node 2: (1, 2)
- from node 1 to node 3: (1, 2), (2, 3)
- from node 1 to node 4: (1, 2), (2, 3), (3, 4)
- from node 1 to node 5: (1, 5)
- from node 1 to node 6: (1, 2), (2, 6)
- from node 1 to node 7: (1, 2), (2, 3), (3, 7)
- from node 1 to node 8: (1, 2), (2, 3), (3, 4), (4, 8)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $\quad Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$; $\quad Q \coloneqq Q \backslash \{i\}$;

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$; $Q \coloneqq Q \backslash \{i\}$;

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$; $Q \coloneqq Q \backslash \{i\}$;

select the first node of list Q and call it «node i»

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$;

$Q \coloneqq Q \backslash \{i\}$;

select the first node of list Q and call it «node i»

delete the selected node from Q

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$;

$Q \coloneqq Q \backslash \{i\}$;

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

select the first node of list Q and call it «node i»

delete the selected node from Q

for each arc (i, j) of the forward star of node i check the current value p_j (predecessor of node j):

- if $p_j = 0$ (i.e. at the current iteration node j is reached for the first time), set $p_j = i$ and insert node j in Q (2 ways of managing list Q)
- if $p_j \neq 0$ (i.e. in a previous iteration node j has already been reached from node p_j by means of arc (p_j, i), do nothing

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

Procedure Oriented paths (G, r, p) :

begin

for $i \coloneqq 1$ **to** n **do** $p[i] \coloneqq 0$;

$p[r] \coloneqq nil$; $Q \coloneqq \{r\}$;

repeat

$i \coloneqq Select(Q)$;

$Q \coloneqq Q \backslash \{i\}$;

for each $(i, j) \in FS(i)$ **do**

if $p[j] \coloneqq 0$ **then begin** $p[j] \coloneqq i$; $Q \coloneqq Q \cup \{j\}$ **end**

until $Q = \emptyset$;

end.

select the first node of list Q and call it «node i»

delete the selected node from Q

if the list is empty
terminate the iterative process

for each arc (i, j) of the forward star of node i check the current value p_j (predecessor of node j):

- if $p_j = 0$ (i.e. at the current iteration node j is reached for the first time), set $p_j = i$ and insert node j in Q (2 ways of managing list Q)
- if $p_j \neq 0$ (i.e. in a previous iteration node j has already been reached from node p_j by means of arc (p_j, i), do nothing

output: vector of predecessors $\boldsymbol{p}$ ($p_i = 0$: node i cannot be reached from node r)

1) Q is a **queue** (fila):

the **children** of the selected node i are **inserted at the back of list** Q and

are extracted from Q only after the last **brother** of i has been extracted

(graph exploration strategy: breadth-first search (bfs) (visita a ventaglio, o in larghezza)

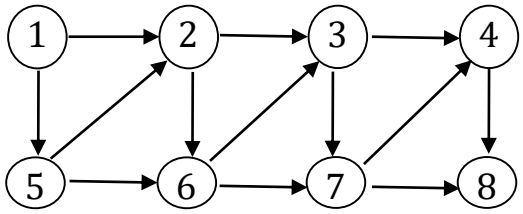
2) Q is a **stack** (pila):

the **children** of the selected node i are **inserted at the top of stack** Q and

are extracted from Q before the **brothers** of i still in Q when node i is extracted

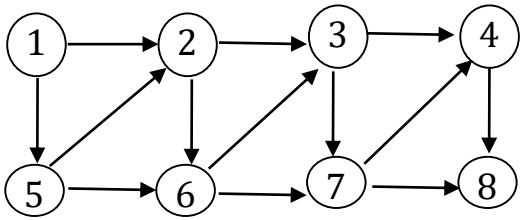
(graph exploration strategy: depth-first search (dfs) (visita in profondità, o a scandaglio)

Version with Q organized as a **stack** (pila) – Initialization



$p_1 = nil \quad p_2 = 0 \quad p_3 = 0 \quad p_4 = 0$

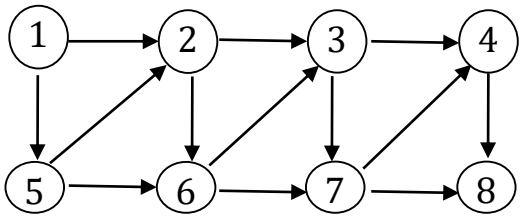
$Q = \{1\}$



$p_5 = 0 \quad p_6 = 0 \quad p_7 = 0 \quad p_8 = 0$

Version with Q organized as a **stack** (pila) – Iteration 1

$p_1 = nil \quad p_2 = 0 \quad p_3 = 0 \quad p_4 = 0$



$p_5 = 0 \quad p_6 = 0 \quad p_7 = 0 \quad p_8 = 0$

$Q = \{1\}$

Selected node: $i = 1 \rightarrow Q = \emptyset$

$FS(1) = \{(1,2), (1,5)\}$

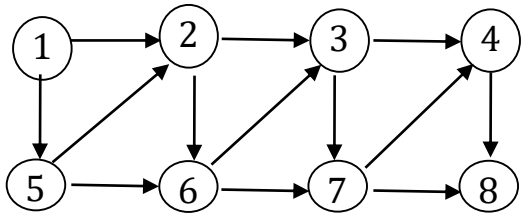
$(1,2)$: $p_2 = 0$

$\Rightarrow \quad p_2 = 1 \qquad Q = \{2\}$

$(1,5)$: $p_5 = 0$

$\Rightarrow \quad p_5 = 1 \qquad Q = \{5,2\}$

$p_1 = nil \quad p_2 = 1 \quad p_3 = 0 \quad p_4 = 0$

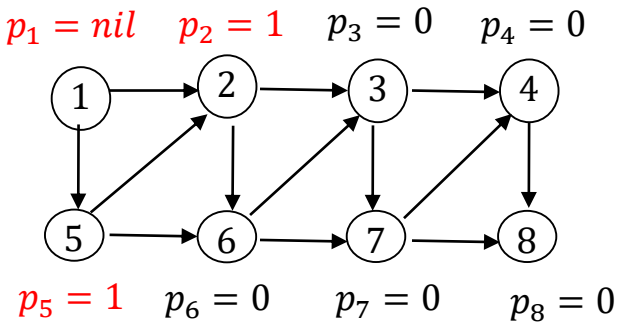


$p_5 = 1 \quad p_6 = 0 \quad p_7 = 0 \quad p_8 = 0$

Version with Q organized as a **stack** (pila) – Iteration 2

$Q = \{5, 2\}$

Selected node: $i = 5 \rightarrow Q = \{2\}$

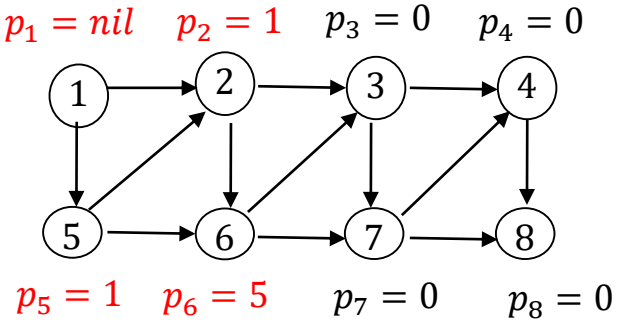


$FS(5) = \{(5, 2), (5, 6)\}$

$(5, 2)$: $p_2 \neq 0$

$(5, 6)$: $p_6 = 0$

$\Longrightarrow \quad p_6 = 5 \qquad Q = \{6, 2\}$



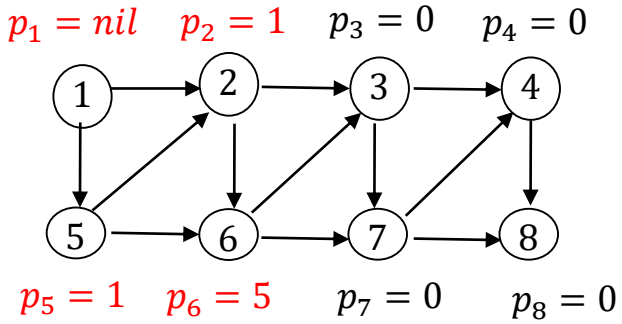
- nods 2 and 5, **children** of 1, are examined in reverse order, since 5 is inserted in Q after 2 and is therefore extracted first

- extracting 5 from Q causes the insertion of 6 at the top of the stack, therefore the exploration continues from nodo 6 (**child** of 5) instead of from node 2 (**brother** of 5)

Version with Q organized as a **stack** (pila) – Iteration 3

$Q = \{6, 2\}$

Selected node: $i = 6 \rightarrow Q = \{2\}$



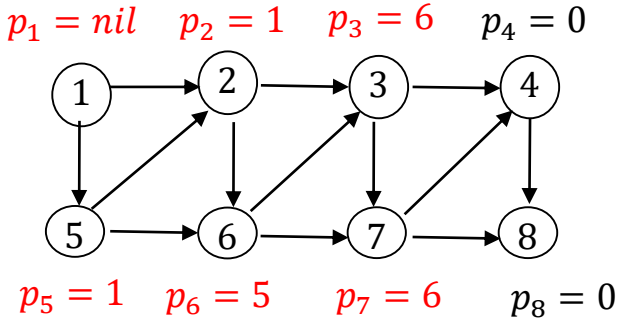
$FS(6) = \{(6, 3), (6, 7)\}$

$(6, 3)$: $p_3 = 0$

$\Longrightarrow \quad p_3 = 6 \qquad Q = \{3, 2\}$

$(6, 7)$: $p_7 = 0$

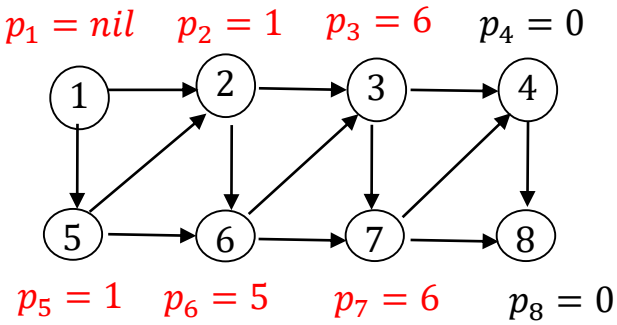
$\Longrightarrow \quad p_7 = 6 \qquad Q = \{7, 3, 2\}$



Version with Q organized as a **stack** (pila) – Iteration 4

$Q = \{7, 3, 2\}$

Selected node: $i = 7 \rightarrow Q = \{3, 2\}$



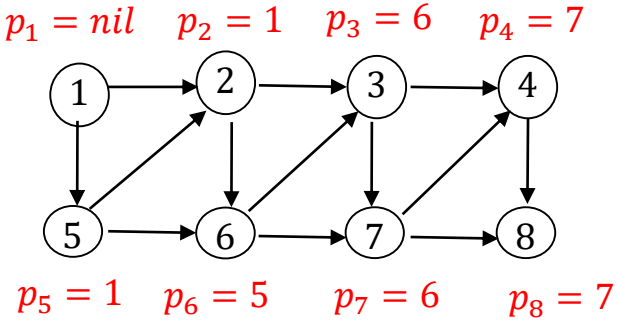
$FS(7) = \{(7,4), (7,8)\}$

$(7,4)$: $p_4 = 0$

$\Longrightarrow \quad p_4 = 7 \qquad Q = \{4, 3, 2\}$

$(7,8)$: $p_8 = 0$

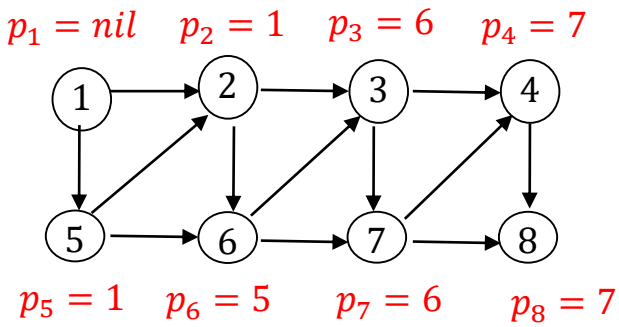
$\Longrightarrow \quad p_8 = 7 \qquad Q = \{8, 4, 3, 2\}$



Version with Q organized as a **stack** (pila) – Iteration 5

$Q = \{8, 4, 3, 2\}$

Selected node: $i = 8 \rightarrow Q = \{4, 3, 2\}$

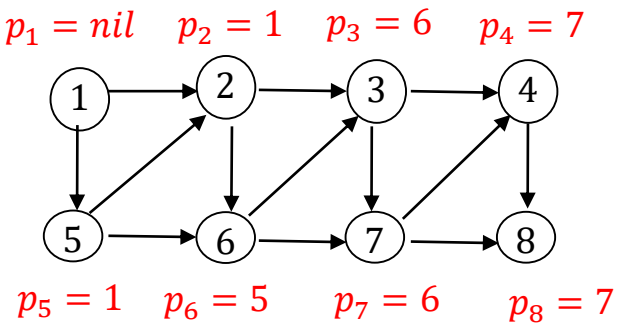


$FS(8) = \emptyset$

Version with Q organized as a **stack** (pila) – Iteration 6

$Q = \{4, 3, 2\}$

Selected node: $i = 4 \rightarrow Q = \{3, 2\}$



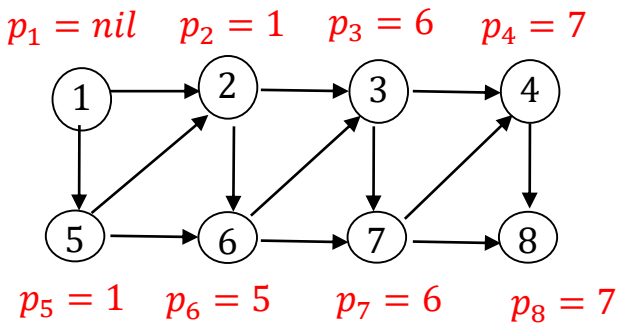
$FS(4) = \{(4, 8)\}$

$(4, 8)$: $p_8 \neq 0$

Version with Q organized as a **stack** (pila) – Iteration 7

$Q = \{3, 2\}$

Selected node: $i = 3 \rightarrow Q = \{2\}$



$FS(3) = \{(3, 4), (3, 7)\}$

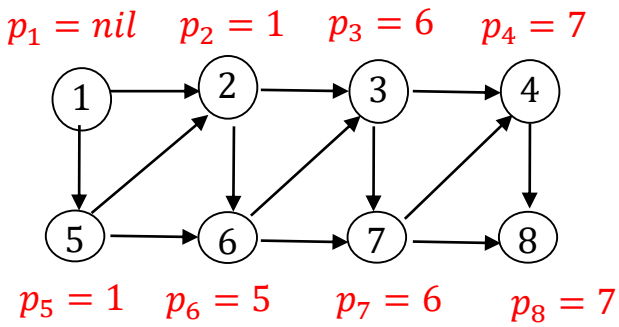
$(3, 4)$: $p_4 \neq 0$

$(3, 7)$: $p_7 \neq 0$

Version with Q organized as a **stack** (pila) – Iteration 8

$Q = \{2\}$

Selected node: $i = 2 \rightarrow Q = \emptyset$



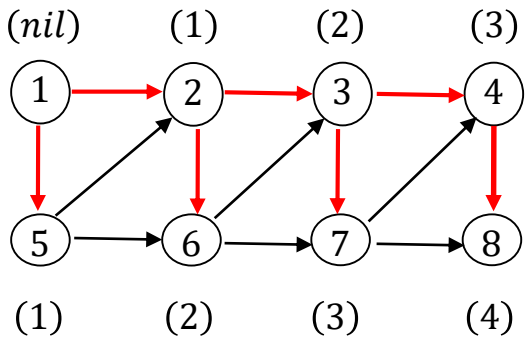
$FS(2) = \{(2,3),(2,6)\}$

$(2,3)$: $p_3 \neq 0$

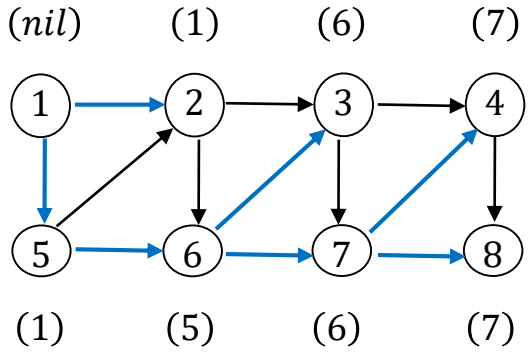
$(2,6)$: $p_6 \neq 0$

$Q = \emptyset$: the algorithm terminates.

Path tree with Q **queue**



Path tree with Q **stack**



	queue	**stack**
from node 1 to node 2	$(1,2)$	$(1,2)$
from node 1 to node 3	$(1,2), (1,3)$	$(1,5), (5,6), (6,3)$
from node 1 to node 4	$(1,2), (1,3), (1,4)$	$(1,5), (5,6), (6,7), (7,4)$
from node 1 to node 5	$(1,5)$	$(1,5)$
from node 1 to node 6	$(1,2), (2,6)$	$(1,5), (5,6)$
from node 1 to node 7	$(1,2), (1,3), (3,7)$	$(1,5), (5,6), (6,7)$
from node 1 to node 8	$(1,2), (1,3), (1,4), (4,8)$	$(1,5), (5,6), (6,7), (7,8)$

Different paths are determined by the two versions; version with **queue** generates longer paths than version with **stack**

The procedure in which Q is implemented as a queue determines,

for each node i reachable from r,

an oriented path from r to i of minimum length in terms of number of arcs

(this property will be used in an algorithm for the Maximum Flow problem)

Complexity analysis:

- node i is inserted in Q only the first time it is reached:

  there will be at most n insertions and removals of nodes from Q

- each arc (i, j) is examined at most once (if and when i is removed from Q)

  the total number of repetitions of the operations performed

  in the 'repeat . . . until' is limited above by m

The complexity of the Procedure is $O(m)$

Determine the set of nodes that can be reached by oriented paths starting from any of the nodes in a given set $R \subset N$
(several distinct root nodes, rather than a single root node):

apply procedure *Oriented paths* to graph $G' = (N', A')$

- $N' = N \cup \{s\}$, with s **dummy node** (super-root)

- $A' = A \cup \{(s, r) : r \in R\}$

The solution for G is obtained from the solutione for G'
by eliminating the first arc (s, r) from each path.

Determine the set of nodes from which destination node d can be reached by means of oriented paths:

apply procedure *Oriented paths* to graph $G' = (N, A')$

- $A' = \{(j, i): (i, j) \in A\}$
- $r = d$

Same nodes as the original graph G and all arcs with opposite direction

The solution for G is obtained from the solution for G'
considering the arcs with their original orientation